



Software Defect Prediction – Using Object Oriented Metrics

Project Thesis

Submitted By

19-39433-1	Tazrian Zaman Tushti
19-39527-1	Tasnim Khondoker Mim
19-40083-1	Md. Abdullah Al Hossain
19-39708-1	Afique Sadique

**Department of Computer Science
Faculty of Science & Technology
American International University Bangladesh**

8th February, 2023

Declaration

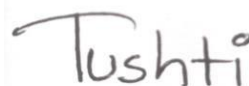
We declare that this thesis is our original work and has not been submitted in any form for another degree or diploma at any university or other institute of tertiary education. Information derived from the published and unpublished work of others has been acknowledged in the text and a list of references is given.



Tasnim Khondoker Mim

19-39527-1

Computer Science and Engineering



Tazrian Zaman Tushti

19-39433-1

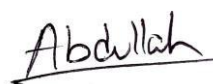
Computer Science and Engineering



Afique Sadique

19-39708-1

Computer Science and Engineering



Hossain Md. Abdullah Al

19-40083-1

Computer Science and Engineering

Approval

The thesis titled “Software Defect Prediction-Using Object-Oriented Metrics” has been submitted to the following respected members of the board of examiners of the Faculty of Science and Technology impartial fulfillment of the requirements for the degree of Bachelor of Science in Computer Science and Engineering on dated and has been accepted satisfactory.

Kazi Sadia

Lecturer & Supervisor
Department of Computer Science
American International University-Bangladesh

Dr. Md. Abdullah - Al - Jubair

Assistant Professor,
Head-In-Charge [Undergraduate Program] &
External
Department of Computer Science
American International University-Bangladesh

Dr. Md. Abdullah – Al – Jubair

Assistant Professor
Head-In-Charge [Undergraduate Program]
Department of Computer Science
American International University-Bangladesh

Prof. Dr. Dip Nandi

Professor & Director
Faculty of Science & Technology
American International University-Bangladesh

Mashiour Rahman

Senior Associate Professor
Associate Dean-In -Charge [FST]
American International University-Bangladesh

Acknowledgement

This thesis is the result of our hard work and our supervisor's motivation. First and foremost, we want to express our gratitude to the Almighty Allah for enabling us to put out the effort necessary to complete this thesis throughout the semester. Special gratitude to honourable supervisor **Kazi Sadia** faculty member, American International University-Bangladesh [AIUB] for her unconditional support, motivation and direction. This thesis project was made feasible by her superb supervision and ongoing assistance. She gave us the chance to collaborate with her, and we are really appreciative. We would like to convey our thanks to our honourable Vice Chancellor, Dr. Carmen Z. Lamagna, our Dean, Prof. Dr. Tafazzal Hossain, our Associate Dean, Mashioor Rahman, our Director, Dr. Dip Nandi, and our Head of Department, Dr. Md. Abdullah- Al - Jubair for their motivation and support. We would like to exhibit our gratitude for giving us the opportunity to participate in this thesis work.

Table of Contents

Chapter 1: Introduction.....	8
1.1 Introduction.....	8
1.2 Motivation	9
1.3 Objective.....	10
1.4 Research Question.....	10
Chapter 2: Literature Review	11
Chapter 3: Methodology	15
3.1 Introduction.....	15
3.2 Proposed Framework	15
3.2.1 Working Procedure	16
3.2.2 Data Collection.....	18
3.2.3 Applied Classifier.....	19
Chapter 4: Analysis and Discussion	22
4.1 Challenges	22
4.2 Comparisons.....	22
4.2.1 Existing Work	22
4.2.2 Classifier Selection	23
4.3 Discussion	25
4.4 Results	25
Chapter 5: Conclusion	29
5.1 Future Work	29
5.2 Conclusion	29
References	30

Table of Figures

Figure 1: Diagram for model construction of SDP	15
Figure 2: Model Accuracy using Coupling metrics	25
Figure 3: Model Accuracy using CK metrics.....	26
Figure 4: Model Accuracy using size metrics	26
Figure 5: Model Accuracy using Pearson correlation	27

Abstract

Software Defect Prediction is an important research area because this prediction helps to ensure quality that will help to reduce cost and effort of software development. An effective SDP model can help the developer's quick detection of bugs. To ensure a bug free software, SQA teams are always trying to improve software quality and SDP can help to determine which module require more testing. Nowadays, Object Oriented programming is being used than procedural programming, so it is vital to evaluate the performance of software using OO metrics than traditional parameters. This research gives the model for OO software defect prediction. These models are capable enough to find out bugs at class level. To predict bugs, different code-based metrics is used and the data is from Eclipse Bug Prediction Dataset. To find out relation strength of different metrics with bugs, Pearson Correlation is applied and model is also created using that. After applying classifiers, it was seen that Logistic Regression outperformed than other classifiers and accuracy was around 84%.

Chapter 1: Introduction

1.1 Introduction

Nowadays the uses of software application are increasing, it has become very important to maintain and ensure its reliability and quality which is essential in software development. It is challenging to create software that is error-free since the complexity and limitations under which they are being built is increasing. Predicting crucial quality attributes like fault-proneness, maintenance effort and reliability in the early phases of software development can be the way to deal this problem [1]. Software developers and testers are unable to do extensive testing to ensure that the software is free from bugs due to their inadequate SQA funds and resources. Besides, the complexity, design and size of software system makes it harder to inspect and review each line of codes. Software Defect Prediction (SDP) has been recognized as one of the most important study areas since the beginning of the software era as it attempts to find out the higher fault occurrence area [2].

Since testing is regarded as one of the important steps of software development life cycle (SDLC) and this testing step is closely connected with software quality so, SDP can play an important role to ensure software quality. When software is released with bugs, significant time and resources are expended to make it error-free. Thus, it is wiser to detect errors as early as possible. Defect prone modules are needed to be identified at early stages so that testers can give more attention which modules need more detailed testing. SDP can help to predict bugs early stage, to determine whether crucial modules require additional resources, can help to ensure software quality and thus, decrease the cost. Because a defect prediction model may give important hints regarding the placement and position of errors, as a result, priority of testing, proper prediction can lower testing expenses and can help to ensure quality of software.

Without being explicitly programmed, machine learning forecasts future events based on past data. It concentrates on the output parameter, tries to find patterns in the data, and forecasts result. Prediction of software defect is an approach which is supervised in which a set of independent parameters (predictors) and a machine learning technique are used to predict the value of the dependent variable (a class's fault-proneness). There are machine learning methods that are applied to anticipate fault-proneness. These algorithms are chosen based on the dataset and the accuracy is also dependent on the dataset. Different classifiers give accuracy differently based on the design and property of the data, how these data are chosen. There are some metrics(attributes) that can be used to forecast defects like: static code metric, change metric, object-oriented metric, size metric etc. It is important to select the metric wisely that will help to capture the quality attributes. An

empirical investigation of actual systems can only provide solutions. Empirical research has been done to determine which metrics affect software quality and to build models that uses those metrics to predict quality attribute [3].

There is not many research on machine learning application for object-oriented based metrics for defects prediction. Statistical techniques are used to create the majority of the prediction models in the research. There are many distinct machine learning techniques, and because they provide varying results, it is essential to compare outcomes of different ML techniques [4]. A number of articles have also evaluated the model's propensity to identify fault-proneness on modules like files, packages-based levels. The majority of machine learning approaches yielded intermediate accuracy rates between 70% and 83%, with increased misclassification errors. [2].

Investigation of the reasons behind such accuracy provided with information that main causes were lack of suitable metrics which can evaluate the capability of these algorithms and the datasets used to train the system were unbalanced. Moreover, further investigation led to a discussion that wrong choices of picking metrics without correlation to the project models could be a reason behind low accuracy.

To continue this research Eclipse dataset has been chosen. The aim of this research is to comprehend connections between a few selected metrics of software with defects at a class level. The aim is to carefully select different metrics and use those on different ML models to observe difference in result in order to see if better results can be achieved in order to find defects.

1.2 Motivation

Software quality is the thing that users, developers want to achieve. But nowadays, the complexity and design of a software is becoming more complex. So, it goes without saying that the more complex the software, the more it is easier to have bugs. So, to ensure the quality of software, it is essential to detect defects as early as possible. Through software defect prediction, it is possible to find defects existence in class. Object-oriented approach is different from procedural as nowadays most of the developers are using object-oriented concepts. So, the constructed model is capable of finding bugs in object-oriented based software. That was the reason of developing the project that will help to ensure quality of software as well as it will help build software's in time and in less cost.

1.3 Objective

This research paper develops a model for software defect prediction using object-oriented metrics.

- The main objective is to assure quality of software in software development and to make it more effective some ML techniques were used to determine whether this can be applicable in real software projects.
- Exploring the relation of object-oriented metrics with defects since all metrics are not a good predictor was also a focus on this research.
- These models will help to reduce the number of bugs at class level.
- Defect predictions can be done through these models more efficiently as a result testing efforts can be distributed more appropriately and thus help testers to determine which modules require more testing.
- Prior detection of bug at one module can also help to determine whether other modules which are dependent on that module may have bugs, as a result defect prediction will help to find out bugs.

1.4 Research Question

The main focus of this research is to construct a bug prediction model based on different metrics and compare different ML methods. In this research the following research questions has been addressed:

1. How different metrics can give accuracy for models and which outperforms most for finding bugs?
2. Are the results accurate enough to apply this process in actual software projects?
3. Which OO metrics are good predictor?

Chapter 2: Literature Review

Sandeep Kumar et al. carried out study on the dynamic selection of leaning technique. SFP techniques haven't consistently provided the highest accuracy in evaluations in any circumstance. This research presents and discusses a method for predicting software faults that relies on selecting the most appropriate learning technique from a collection of competing ones in order to create a fault prediction model. The suggested SEP approach is used to analyze the results of nine object-oriented (OO) project datasets and five datasets from Eclipse projects. The cost analysis is described extensively. All of the OO datasets are accessible to the general public in the PROMISE data repository and are comparable to the open-source Apache task. 21 software program measures that are similar to the many OO paradigm attributes were present in the nine OO datasets used in the study. The performance of every approach is assessed for all Eclipse datasets and OO project datasets. All of the datasets in the Eclipse project dataset have AUC values higher than 0.65, with the greatest value being 0.78. AUC values are always greater than or equal to 0.60 for datasets from OO projects, with the highest value being 0.825. The software defect prediction was successfully completed using the method that was described.[2]

Yogesh Singh et al. empirically validated the object-oriented metrics for estimating fault proneness models. The primary objective of this work was to determine the relationship between object-oriented metrics and fault proneness at various fault severity levels. Several other models have been constructed using regression and machine learning techniques. Chidamber and Kemerer studied and compared all the methods for various OO metrics fault severity levels. In this research, various methods, including decision trees (DT) and artificial neural networks, as well as linear and logistic regression methods, are proposed. Faults were divided into high and low severity categories by Zhou and Leung (2006). Fault proneness is the probability that a class will include a flaw. There were less errors, and out of 145 classes, 59 classes had zero or one error. Low DIT and NOC values are present in the system. ROC analysis was carried out throughout the research procedure. For the CBO, WMC, RFC, and SLOC measurements, it was judged that all fault severity levels were considerable. The LCOM statistic was not regarded as crucial in relation to the LSF. Recreating this study will help predict machine learning systems like genetic algorithms.[4]

Dr. M.P. Thapaliyal et al. examined two metrics from the Chindamber and Kemerer matrices suite, Weighted Method per class (WMC) and Coupling between Object Classes (CBO). To determine the type of connection between these metrics and faults, an empirical investigation was conducted. 50 Java class examples from various projects are provided. Three hypotheses have been formed according to the study. For this research, only projects that utilized Java as their major development language were chosen. While WMC and CBO are collected from the documentation and source codes, Size Metric is based on the information provided by the students. Weighted least squares were used to recover the heteroskedasticity problem. According to El Emam et al. (2001), size does complicate the impacts of metrics on faults. Researchers tested every correlation with several projects and discovered some discrepancies in the findings. While WMC and CBO have been found to have a negligible impact on flaws, class size has been found to have a positive and considerable impact. [5]

This research describes that in order to analyze a more limited collection of metrics, Marian JURECZKO et al. presented a new version of the tool that significantly expanded earlier versions. The fundamental objective is to build software defect prediction models, with ckjm calculations serving as the model input. 20% of files, according to Weyker et al., have up to 80% faults. 16 software initiatives were examined. The metrics and defect data for software projects have been gathered from source code archives. The metrics have been computed using the ckjm tool. To find flaws, another program named BugInfo has been created. This research also investigates whether class size is a relevant factor in the defect prediction models. WMC (Weighted Methods per Class) and LOC are two metrics that have been taken into consideration as class size factors (Lines of Code). The LOC based models were clearly superior to the WMC based models as basic models, but neither the WMC based models nor the LOC based models had statistically significant differences.[6]

In the paper the lists of pre- and post-release defects in each package and file of the Eclipse dataset were studied by Thomas Zimmermann et al. The Eclipse programming environment's code base has been the topic of work. In the first six months both before and after release, there were a recorded number of defects. In this study, researchers added counts of syntactic components and common complexity metrics to the data. Additionally, OO measures that are helpful in predicting defect density were validated by Basili et al. Eight further empirical experiments by Subramanyam and Krishnan demonstrated the correlation between OO metrics and faults. Design data can predict post-release issues, as Schroter et al. found. The average, total, and maximum values of the metrics were left out because they are typically 0 when using Eclipse's Java parser. According to the findings of the experiment presented in this research, a combination of

complexity metrics can predict the presence of more faults, and complicated code tends to include more of them. [7]

Cataldo et al. proposed a statistical approach to generate a regression model. Only 4 out of 34 code and process metrics, according to a case study on Eclipse open-source, have an impact on the identification of a post-release problem. This research shows that a substantial amount of research has centered on locating the software's fault-prone areas. The primary goal was to utilize code and process metrics that had previously been used in complex prediction models, and it was focused on a lower number of logistic regression models to understand the effects that post-release problems have. Cataldo et al. approached statistical method was significant and had few collinear measures that affected the post-release problems. For three different Eclipse releases, the logistic regression models were developed to assess all the independent variables. All the approaches were successful but there were few questions that lingered till future work.[8]

In order to determine the connection between object-oriented metrics provided by Chidamber et al. defect proneness at various levels, this research describes how to construct an SVM model. By using receiver operating characteristic (ROC) analysis, this SVM approach was validated. Since complexity and the constraints are increased its impossible to find software without faults. The inputs of SVM methods are process and product metrics which helps the outputs providing information about the software quality. The effectiveness of the machine-learning technique to determine how well SVM could predict fault propensity in the classes. OO metrics suites are used to determine how accurately fault proneness will be predicted. Three levels of flaws were identified: high, medium, and low. These metrics were taken to analyze coupling, cohesion, inheritance and size metrics for the classes. Out of 23 classes 16 classes were evaluated to be fault prone. The findings showed the usefulness of NOC and DIT metric is poor. This research's findings have been repeated in several studies using different datasets. [9]

Using object-oriented metrics, Dharmendra Lal Gupta et al. conducted research on anticipating software defects. There was a model developed for an object-oriented software bug prediction system. When a new system is evaluated on a class, this model predicts when bugs will exist in that class. Hypotheses were formulated corresponding to each and every metrics. Among these, logistic regression has the best classifier accuracy. OO approach is different from the traditional programming approach. Better dependability, maintainability, and reusability are all benefits of this approach. In this research, about 12 open-source applications of a similar nature were chosen. But the proposed model only considers OO datasets. 14 different classifiers were applied including Naïve Bayes, SVM, Logistic regression etc. The proposed model provides an average of 76.27% accuracy at testing stage after the all the datasets are trained.[10]

This research paper machine learning algorithms to predict software processes. Three Eclipse releases (2.0, 2.1, 3.0), an open-source Java development tool project, were applied in the studies. On both static code and change measurements, as well as the combination of both sets, a machine learning approach was applied. The results overlapped features which was found in the two studies. This research's objective is to explain the performance of simpler models with fewer characteristics. All the algorithms were applied and the results were later compared. Tenfold cross validation was used for machine learning. Reduced set of metrics produces higher accuracy than Naïve Byes and J48 algorithms where all the change metrics were used. The experiment, which showed that using change metrics is equal than using static code metrics, was also repeated. The G score, however, performs better when all measures in the set are changing. [11]

Using supervised machine learning techniques, S. Delphine Immaculate et al. published a study on bug prediction. Three machine learning algorithms are used to develop the model and estimate the occurrence of software faults based on past data. In the study of bug prediction, logistic regression, naive bayes, and decision tree algorithms have been taken into consideration. These In order to predict upcoming software bugs, supervised machine learning methods are combined with classifier deployments to improve the predictive accuracy of the models. The predictive technique of logistic regression is similar to an object that belongs to one class or another. Categorical and continuous input and output variables can both be used with decision trees and random forests. Prior to using the methods, all the variables were pre-processed. The proposed model in this research, which was built utilizing individual classifiers, has a tendency to be less accurate. Any method can be used to detect bugs, however due to its ensemble nature, random forest is recommended. The use of artificial neural networks will improve the accuracy of future forecasts. While some algorithms can produce results with 100% accuracy, ANN is thought to be a preferable alternative for future research. [12]

Chapter 3: Methodology

3.1 Introduction

Firstly, data should be collected. For data collection different OO software projects can be used. In this research, Eclipse dataset has been chosen. After data collection, data should be pre-processed as there might be missing values or irrelevant values. When dataset gets ready, different metric is chosen. There are basically 3 types of metrics in OO concepts. Pearson correlation is applied to find the relation of different parameters with bugs so that those metrics can help to find out bugs more effectively. Then the dataset is split into test and training set for the model. Then classifiers are applied to train the models and test them so that they can provide good accuracy as a result, bug can be predicted for that class.

3.2 Proposed Framework

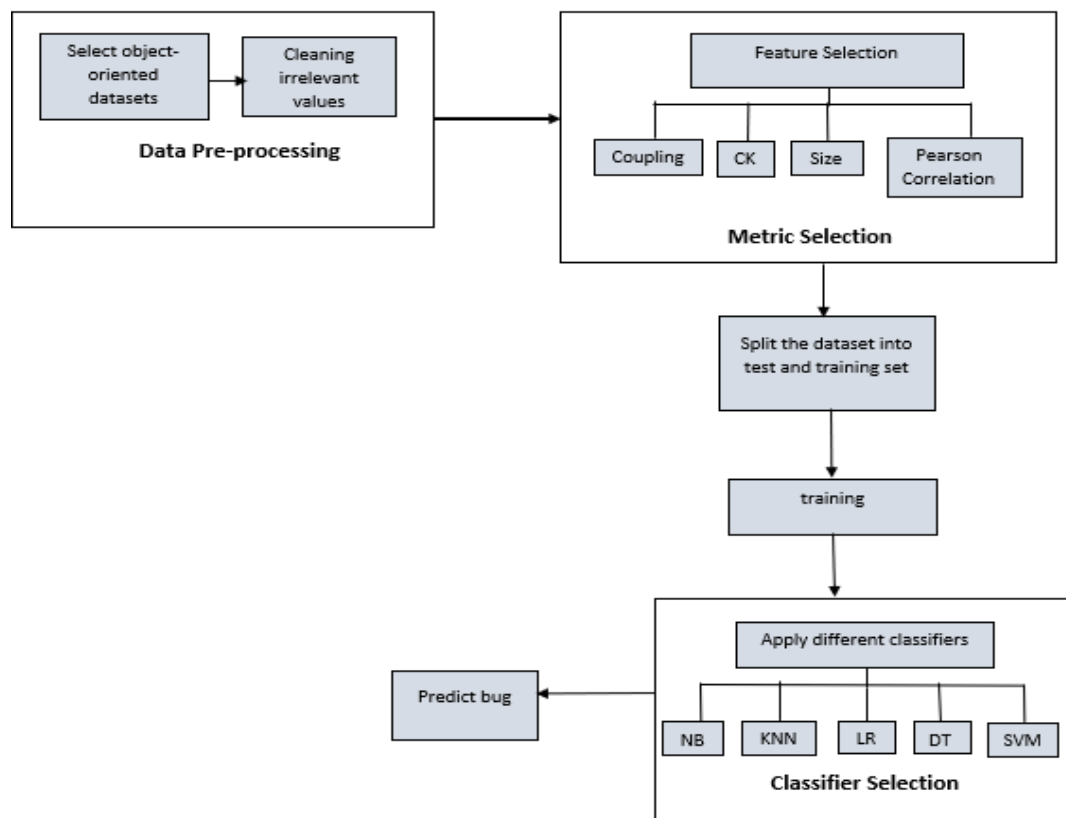


Figure 1: Diagram for model construction of SDP

3.2.1 Working Procedure

Data collection & pre-processing:

Software that are made based on OO programming can be used. So, data should be collected which are OO based. After data collection, the data should be pre-processed. It should be checked whether any parameter has any missing values or not. This research was based on Eclipse dataset. This dataset has some irrelevant parameters as our research focus is binary based prediction whether module has bug or not. But categorizing the bugs is not the concern here as a result those parameters are excluded for this research.

Metric Selection:

In the dataset there are 17 metrics (independent variable) and one bug class (dependent variable) was found. This dataset contains three types of metrics which are: size, coupling and CK metric.

Size metric: Size oriented metrics gives the information about the size of the program. There are 9 size metrics. The size metric is: Number of attributes (NOA), Number of public attributes (NOPA), Number of private attributes (NOPRA), Number of attributes inherited (NOAI), Number of lines of code (LOC), Number of methods (NOM), Number of public methods (NOPM), Number of private methods (NOPRM), Number of methods inherited (NOMI).

Coupling metric: Metric for coupling express how dependent software modules or classes with one another. There are 2 coupling metrics: fan-in, fan-out.

CK metric: Classes, inheritance, and encapsulation are properties of OO systems that are measured using the CK metric. 6 CK (Chidember, Kemerer) are: Weighted Methods per Class (WMC), Depth of Inheritance Tree (DIT), Number of Children (NOC), Coupling Between Objects (CBO), Response for a Class (RFC), and Lack of Cohesion in Methods (LCOM).

To find the relation among metrics with bug, Pearson Correlation was applied. Then classifiers were applied to predict bug.

Pearson Correlation is the way to measure the relationship of variables. It has a value ranging between -1 to +1 where 0 means there is no relation, +1 means positive relation between two parameters, if value of one variable increases other one also goes up and -1 means if value of one variable increases, other one goes down. In our dataset Pearson Correlation was applied at $\alpha = 0.01$ level to find out the relation of every metric with bugs separately. The value of r and p are given below:

Metric name	<i>r</i>	<i>p</i>
Coupling between objects	.494	<0.001
Depth inheritance tree	-0.44	.163
Fan-in	.341	<0.001
Fan-out	.506	<0.001
Lack of cohesion in methods	.324	<0.001
Number of children	.055	.083
Number of attributes	.265	<0.001
Number of attributes inherited	.060	.058
Number of lines of code	.591	<0.001
Number of methods	.458	<0.001
Number of methods inherited	-.018	.562
Number of private attributes	.221	<0.001
Number of private methods	.418	<0.001
Number of public attributes	.175	<0.001
Number of public methods	.373	<0.001
Response for a class	.575	<0.001
Weighted methods per class	.581	<0.001

Table 2: Most influential metrics for bug prediction

From table 2, the metrics that are highly related with bugs based on r and p values were selected. The metrics that have higher r value and p is less than 0.001 since $\alpha = 0.01$ was selected. r tells the relationship strength between two variables and p provides the significance. So, cbo, fan-out, lcom, nloc, nopm, rfc and wmc metrics were chosen.

Further processing:

After metric selection, dataset is splitted into training and test set. For that 10-fold cross-validation is applied. 30% is for training the model and 70% for testing.

Classifier Selection: 5 classifiers: NB, DT, LR, SVM, KNN are applied to detect bug.

3.2.2 Data Collection

For this research, there was a need of a software development dataset. Eclipse JDT core dataset was utilized. Data is publicly available on Bug Prediction dataset repository. The link of the dataset is: <https://bug.inf.usi.ch/download.php>. Data from 5 Java programs were retrieved for this bug prediction dataset from class level. There are many datasets that are nowadays are used for similar kind of projects. Bug Prediction dataset provides bug data for 5 listed software systems. The number of pre- and post-release defects was calculated from the following key data sources: Bugzilla, Jira, CVS and SVN. The addition of change measurements to the source code metrics might enhance the effectiveness of failure prediction techniques. Bug Prediction dataset provides data under categories like code investigations and bug. Metrics for products and changes (processes) are gathered by the Bug Prediction Dataset. The related production and procedure metrics at the class level were created by the authors (D'Ambros). They computed several extra object-oriented metrics in addition to the traditional C&K metrics. Prediction models gets constructed from historical data. As a result, these prediction models are particular project based. As a result, these metrics gained from one project usually are not be applicable to other projects rather similar projects will be applicable. Eclipse JDT core dataset is helpful to find predictors that can be used to similar projects to predict defects. Description of metric is given below:

Name of metric	Data type	Description
number of attributes	Numeric	Total number of attributes defined in one class.
number of public attributes	Numeric	total number of public attributes presented in one class.
number of private attributes	Numeric	total number of private attributes presented in one class.
number of attributes inherited	Numeric	Total attributes that a class obtains from its parent class.
number of lines of code	Numeric	Sum of fields, methods and instructions in every method of class.
number of methods	Numeric	Total number of methods defined within one class.
number of public method	Numeric	Total of all methods within one class that are declared as public.
number of private method	Numeric	Total of all methods within one class that are declared as private.
number of methods	Numeric	Total of methods that a class obtains from its

inherited		parent class.
weighted method per class (WMC)	Numeric	It tells how complicated a class is. The WMC has a value equal to the quantity of methods in one class.
depth of Inheritance tree (DIT)	Numeric	DIT metric is the greatest path length from a node to the trees' root node. It becomes more complex when a class is in lower hierarchy as it will obtain more methods and fields from other class.
number of Children (NOC)	Numeric	The number of immediate children classes derived from its parent class. NOC gauges the breadth of a class ranking.
coupling between objects (CBO)	Numeric	Coupling means how two classes are connected or dependent with each other by using methods or instances of other class.
response for a class (RFC)	Numeric	RFC metric counts the variety of methods that may be called when an object of that type gets a message.
lack of cohesion in methods (LCOM)	Numeric	Count sets of methods in a class that are not related by sharing of some of the common class variables.
Fan- in	Numeric	other classes that reference that particular class
Fan-out	Numeric	Number of other classes referenced by that particular class
Bugs	Numeric	Whether that class has bug or not

Table 1: Metrics details

3.2.3 Applied Classifier

Naïve Bayes (NB): One of the most simple and efficient classification algorithms is the naive bayes classifier, which helps in the creation of simple machine learning models that can make quick predictions. It is a probability-based classification method that makes predictions based on an item's likelihood.

Advantages of Naïve Bayes (NB):

- Naïve Bayes doesn't bear any training data
- It is basic and precise to use.
- Proceeding and can be used for real-time predictions

- It can handle both nonstop and separate data

Logistic Regression (LR): Logistic regression is principally performed for prediction and classification problems. The variables are substantially dependent and categorical which provides a probabilistic value between 0 to 1. It is used to classify observations based on numerous forms of data and quickly identify the most effective factors for classification.

Advantages of Logistic Regression:

- One of the simplest machine learning methods is logistic regression, and it is simple to use.
- Trained data can be updated to return new data
- When the dataset contains features that can be separated linearly, logistic regression has been shown to be particularly effective.

K Nearest Neighbor (KNN): It is a basic machine learning algorithm that makes use of the supervised learning approach. The KNN algorithm stores all accessible data and groups input data into categories based on similarity. Newer data can therefore be swiftly categorized into the appropriate category using the KNN approach. The KNN algorithm simply stores the dataset during the training phase and then classifies incoming data into a category that is very close to the previously stored dataset.

Advantages of K Nearest Neighbor (KNN):

- Both classification and regression issues can be solved using KNN.
- KNN classifier updates can be done for less expense.
- Multi-class cases can be handled naturally by it.

Decision Tree (DT): Classification and regression issues can be solved using the supervised learning method known as the decision tree. They are used most frequently, nonetheless, for classification-related problems. It is a classifier with a tree-like structure, where internal nodes stand in for the dataset's features, branches stand for the decision-making steps, and leaf nodes represent the results.

Advantages of Decision Tree (DT):

- Less work is required to prepare the data during pre-processing
- Technical teams and stakeholders can understand the decision tree model very easily.
- Data normalization is not necessary for decision trees.

Support Vector Machine (SVM): The Support Vector Machine, sometimes known as SVM, is one of the most widely utilized supervised learning algorithms. It is applied to problems involving classification and regression. The SVM algorithm's goal is to produce the best decision boundary or line for categorizing n-dimensional space, enabling us to efficiently assign the new data point to the proper category in the future.

- Support Vector Machine is more memory efficient
- As there is no noise and the target class is not overlapping, it performed incredibly well with the data set.
- It provides better accuracy than other classifiers.

Chapter 4: Analysis and Discussion

4.1 Challenges

During this research there were several challenges. This research was conducted on the eclipse dataset. At first, collecting data for this project was quite difficult since there are not enough repositories that could provide such data but at last, we found it out. Preprocessing the dataset was also challenging part of this research. In the dataset, parameters: nonTrivialBugs, majorBugs, criticalBugs, highPriorityBugs contained the value zero (0). These parameters were eliminated from the dataset for applying machine leaning algorithms. There were also some duplicate entries which was deleted and kept in one entry. After that we have applied five algorithms on python where we faced some errors. Those errors were rechecked. All the algorithms had a decent accuracy around 80%. All combinations have been applied but it was not possible to merge the accuracy into 90%.

4.2 Comparisons

There are many similar types of studies that exist that will be described in existing work and best classifier selection will be described later section according to our study.

4.2.1. Existing Work

In this research, they tried to present SFP (Software Fault Prediction) by using different ML techniques by selecting some set of features on Eclipse project dataset and some other OO projects. From their study they found out the accuracy which is 83.5%. For other OO projects, the highest accuracy was 81.5% [2]. This research used the data of Nasa where they validated only CK metrics. They used Support Vector Machine algorithm and the value of AUC was .8[4]. This research also used some open-source software to predict defects at class level. They have used regression model and the accuracy was around 80% [6]. This research also focused on OO projects and data was collected from Promise Software Engineering Repository. They emphasized on CK metrics and the applied algorithm was Logistic Regression and the accuracy was average 76.27% [10]. This study also used Nasa dataset to find out defects, they used LR, DT and ANN. They gave the result of their model based on AUC analysis and result is respectively 0.753, 0.835, 0.809 [13]. So, most of the research focused on CK metrics to predict defects at class level. The performance measurement was different for different researches.

4.2.2. Classifier Selection

In this research Eclipse JDT dataset has been selected where bugs are selected at a class level. This research contains OO which are the code base metrics. There are in total 17 metrics in this dataset and these 17 metrics includes three types of metrics: CK, Coupling and Size. Another set was all the combinations of every attributes. Additionally, Pearson correlation was used. Five classifiers of machine learning algorithms were selected. The classifiers are: Naïve Bayes (NB), K-Nearest Neighbor (KNN), Logistic Regression (LR), Decision Tree (DT) and Support Vector Machine (SVM).

Results for CK:

The whole dataset contains 6 CK metrics (Weighted Methods per Class (WMC), Depth of Inheritance Tree (DIT), Number of Children (NOC), Coupling Between Objects (CBO), Response for a Class (RFC), and Lack of Cohesion in Methods (LCOM)). After applying the classifiers, the following accuracy were resulted for each:

Naïve Bayes (NB):	85.33 %
K-Nearest Neighbor (KNN):	83.33%
Logistic regression (LR):	86%
Decision Tree (DT):	81%
Support Vector Machine (SVM):	83%

Here Logistic regression contains the highest accuracy among all the classifiers. Logistic regression algorithm out performs among the CK metrics

Results for Coupling: Among 17 metrics of the dataset coupling matrices are only 2 of them which are considered coupled. They are: Fan-in, Fan-out. After applying the classifiers on these coupling metrics, the following accuracy were resulted for each:

Naïve Bayes (NB):	83.67 %
K-Nearest Neighbor (KNN):	79.33%
Logistic regression (LR):	85%
Decision Tree (DT):	80.67%
Support Vector Machine (SVM):	83%

Considering the coupling metrics here logistic regression contains the highest accuracy and surpasses among all the classifiers.

Results for size: The remaining 9 metrics are the size metrics (Number of attributes (NOA), Number of public attributes (NOPA), Number of private attributes (NOPRA), Number of attributes inherited (NOAI), Number of lines of code (LOC), Number of methods (NOM), Number of public methods (NOPM), Number of private methods (NOPRM), Number of methods inherited (NOMI)). Following accuracy resulted after applying each of the classifiers:

Naïve Bayes (NB):	83.67 %
K-Nearest Neighbor (KNN):	86.33%
Logistic regression (LR):	84.33%
Decision Tree (DT):	82.33%
Support Vector Machine (SVM):	85.67%

For size metrics K-Nearest neighbor receives the highest accuracy among other classifiers.

Results for Pearson Correlation: Finally, Pearson correlation was applied additionally to see the results of each of the classifiers. The results conclude with these following accuracies:

Naïve Bayes (NB):	84.67 %
K-Nearest Neighbor (KNN):	82%
Logistic regression (LR):	85%
Decision Tree (DT):	76%
Support Vector Machine (SVM):	84%

For Pearson correlation logistic regression has the highest accuracy.

Finally, after analyzing all the 5 classifiers Logistic Regression outperforms among all of them. Among 5 the categories of the metrics Logistic regression contains the highest accuracy in three of them. The second highest accuracy is provided by K-Nearest

neighbor. Since Logistic regression contains the majority accuracy it is considered as the best classifier among all the classifiers.

4.3 Discussion

Throughout the research five classifiers has been used. Each of the classifiers of every metrics gave an accuracy around 82%. Highest accuracy was secured by logistic regression classifiers. So, it is considered as the best performer among other classifiers. K-Nearest Neighbour also can be prioritized after Logistic regression. It has become challenging to upgrade all the accuracy up to 90% which will be worked on future. There are many models on predicting defects hence this work is aimed to create a model to make focus on the object-oriented approaches. This research overall ensured a quality of software which will help to build in a lower cost. This will create a great impact in future industrial development.

4.4 Results

Accuracy of different classifiers are given below:

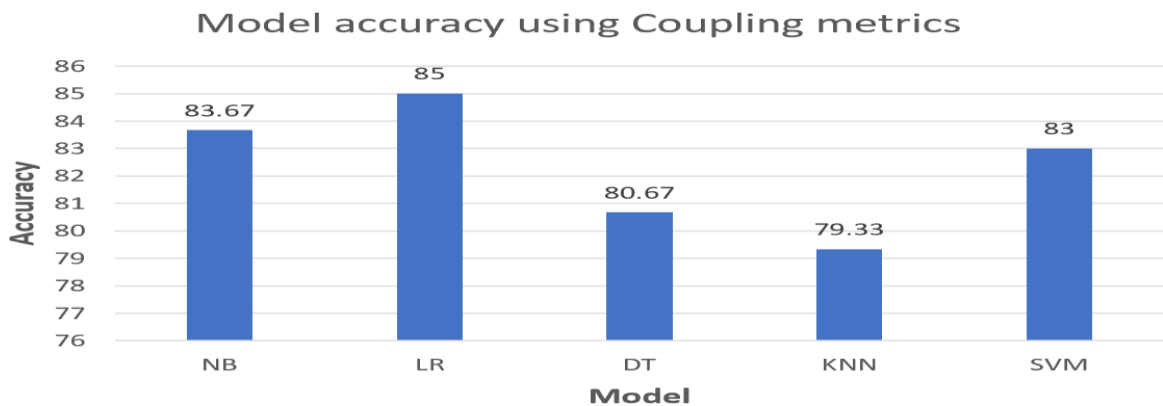


Figure 2: Model Accuracy using coupling metrics

This figure gives the result for different classifiers for coupling metrics. So, from the graph it can be clearly seen that LR outperforms the most for defect prediction. Lowest performed classifier is KNN.

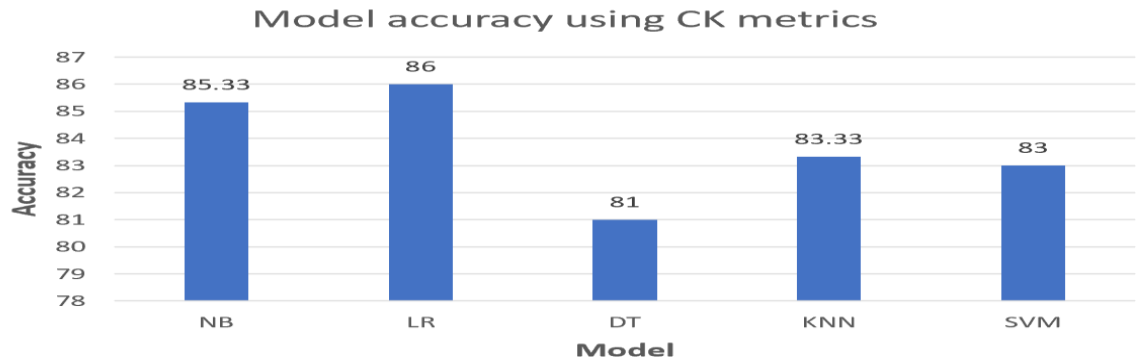


Figure 3: Model accuracy using CK metrics

This figure gives the result for different classifiers for CK metrics. So, from the graph it can be clearly seen that LR outperforms the most for defect prediction and it has the highest accuracy which is 86%.

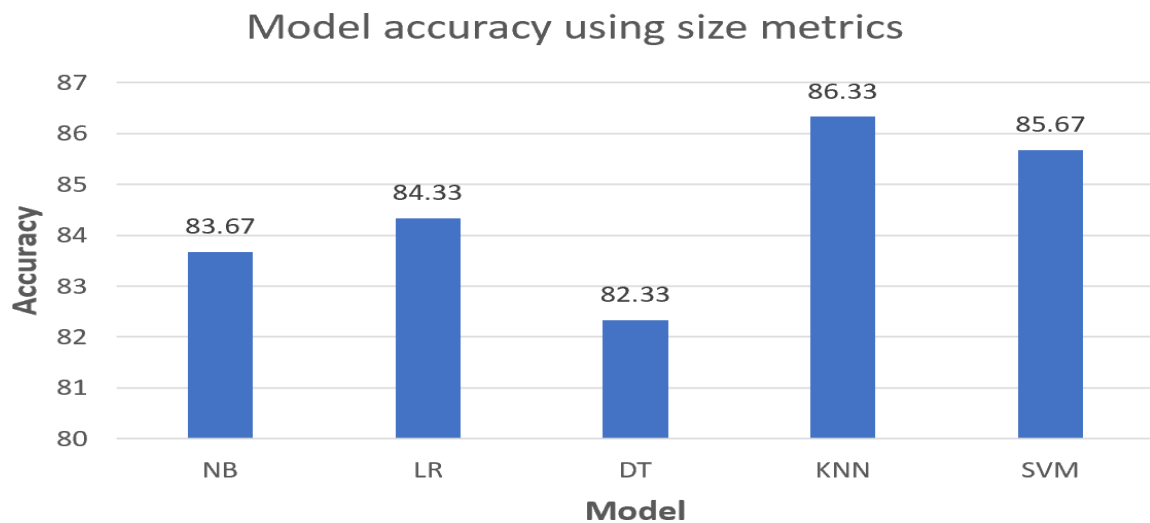


Figure 4: Model accuracy using size metrics

This figure gives the result for different classifiers for size metrics. Here KNN gives good accuracy which is 86.33%

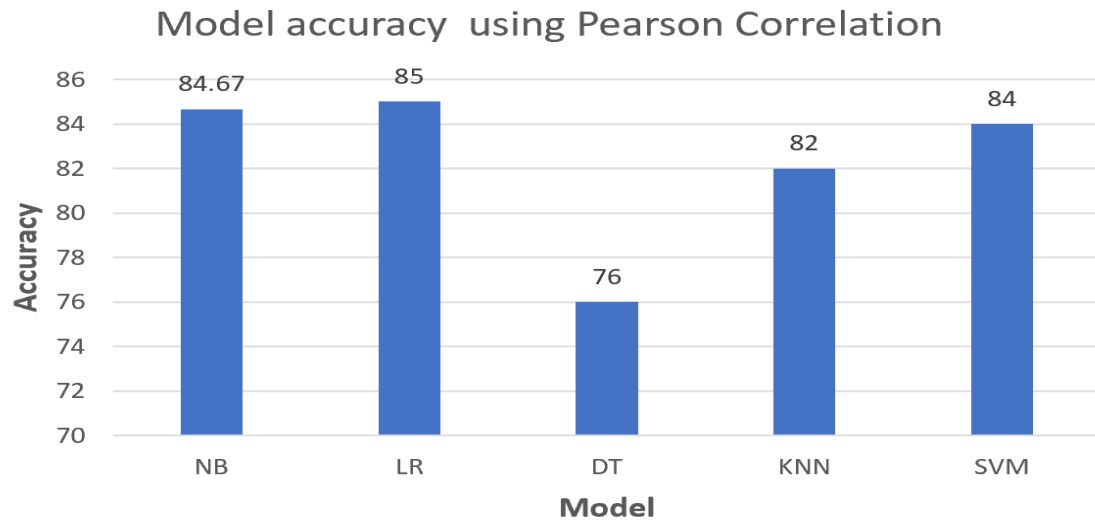


Figure 5: Model accuracy using Pearson Correlation

This figure is all about the accuracy after applying Pearson Correlation. Here LR performs the best among all other classifiers. Since LR classifier has the capability to find out the good indicators and the direction of association, that's why it can provide good accuracy.

So, it is quite evident that LR can be the best classifier for prediction. With Pearson Correlation most relevant metrics are used as predictor for bug prediction.

RQ1 How different metrics can give accuracy for different models and which outperforms the most for predicting bugs?

Answer: From the above graph, the accuracy of models for different metrics are given. Most researches only considered CK metrics rather than size and coupling. This research considered all metrics and it is seen that the accuracy is slightly same for all metrics and LR outperforms in most of the metrics. When Pearson Correlation was considered to find out the relation among metrics with defects, how strongly they are connected, it also proved that LR outperform from all other classifiers.

RQ2 Are the results accurate enough to apply this process in actual software projects?

Answer: The average of accuracy is around 84% for most of the classifiers, there is higher probability of misclassification of defects. If these models are applied in real software projects there is a high risk of getting inaccurate results which can be hindrance

to provide quality assurance. As a result, more studies on this research are needed that can provide around 100% accuracy to ensure the quality of software.

RQ3 Which OO metrics are good predictor?

Answer: After applying Pearson Correlation, this research found out some important metrics as good predictors for finding defects from our datasets. They are: cbo, fan-out, lcom, nloc, nopm, rfc and wmc (these metrics includes size, CK and coupling metrics).

Chapter 5: Conclusion

This chapter explains the overall work progress throughout the research and future possible works to enlighten this research has also been covered throughout this chapter.

5.1 Future Work

The objective of this study was to create a bug prediction model on different metrics and compare different types of algorithms. In order to determine how it will function, it is intended to evaluate more advanced models in the future. There are also plan of implementing deep learning algorithms in this particular dataset to see the whether the accuracy increases or not. In order to further this research, it is planned that more approaches for the control parameters will be explored, such as applying multiple distance measures and procedures.

5.2 Conclusion

Predicting bugs has been an eye-catching moment for researchers as it helps to reduce human effort. These days object-oriented has become an essential part of software environment. It is important to find out the relation of different metrics with defects in software development process. After a software is developed, it is essential to find out the modules that require testing whether it is working properly or not. The major purpose of this research is to explain the performance of less complex models with limited number of features which has been successfully implemented. In this study, object-oriented metrics were utilized to predict class-level faults. Each metrics plays a significant role in detecting the defects. Pearson correlation also played a vital role in finding the relationship of the variables. Using Machine Learning Techniques Logistic Regression has shown the best performance and contains the highest amount of accuracy among majority metrics. K-Nearest Neighbor also plays a major role in accuracy. K-Nearest Neighbor contains the highest accuracy after logistic regression. If other algorithms are applied in future there is a high possibility that accuracy will be upgraded. Using Deep learning will also help to update the accuracy which will be considered in future as well.

References

- [1] A. K. ., Y. S. Ruchika Malhotra, “Empirical validation of object-oriented metrics for predicting fault proneness at different severity levels using support vector machines ,” The Society for Reliability Engineering, Quality and Operations Management (SREQOM), India and The Division of Operation and Maintenance, Lulea University of Technology, Sweden 2011, 2011.
- [2] S. K. Santosh S. Rathore, “Software fault prediction based on the dynamic selection of learning technique: findings from the eclipse project study,” The Author(s), under exclusive licence to Springer Science Business Media, LLC, part of Springer Nature 2021, 2021.
- [3] Y. S. A. K. a. R. M. K. K. Aggarwal, “Empirical Analysis for Investigating the Effect of Object-Oriented Metrics on Fault Proneness: A Replicated Case Study Research,” University School of Information Technology, GGS Indraprastha University, Delhi 110403, India, 2008.
- [4] A. K. ., R. M. Yogesh Singh, “Empirical validation of object-oriented metrics for predicting fault proneness models ,” Springer Science+Business Media, LLC 2009, 2009.
- [5] G. V. Dr. M.P. Thapaliyal, “Software Defects and Object-Oriented Metrics - An Empirical Analysis,” International Journal of Computer Applications , 2010.
- [6] D. D. S. Marian JURECZKO, “Using Object-Oriented Design Metrics to Predict Software Defects,” Oficyna Wydawnicza Politechniki Wrocławskiej, 2010.
- [7] R. P. A. Z. Thomas Zimmermann, “Predicting Defects for Eclipse,” Third International Workshop on Predictor Models in Software Engineering, 2007.
- [8] Z. M. J. W. M. I. B. A. a. A. E. H. Emad Shihab, “Understanding the Impact of Code and Process Metrics on Post-release Defects: A Case Study on the Eclipse Project,” Software Analysis and Intelligence Lab, 2010.
- [9] R. M. • A. K. • Y. Singh, “Empirical validation of object-oriented metrics for predicting fault proneness at different severity levels using support vector machines,” The Society for Reliability Engineering, Quality and Operations Management (SREQOM), India and The Division of Operation and Management, 2011.
- [10] D. L. G. a. K. SAXENA, “Software bug prediction using object-oriented metrics,” Indian Academy of Sciences, 2016.
- [11] K. G.-P. ., D. G. D. a. T. D. Yasser Ali Alshehri, “Applying machine learning to

- predict software fault proneness using change metrics,static code metrics, and a combination of them,” 2018 IEEE, 2018.
- [12] M. F. B. ., M. F. S. Delphine Immaculate, “Software Bug Prediction Using Supervised Machine Learning Algorithms,” 2019 IEEE, 2019.
- [13] R. F. . Z. T. . L. . S. . T. Gyimothy, “A public unified bug dataset for java and its assessment regarding metrics and bug prediction,” The Author(s), 2020.
- [14] A. C. Praman Deep Singh, “Software defect prediction analysis using machine learning algorithms,” 2017 7th International Conference on Cloud Computing, Data Science & Engineering - Confluence (Confluence), 2017.
- [15] W. P. ., G. S. Raimund Moser, “A Comparative Analysis of the Efficiency of Change Metrics and Static Code Attributes for Defect Prediction,” ICSE’08, 2008.